

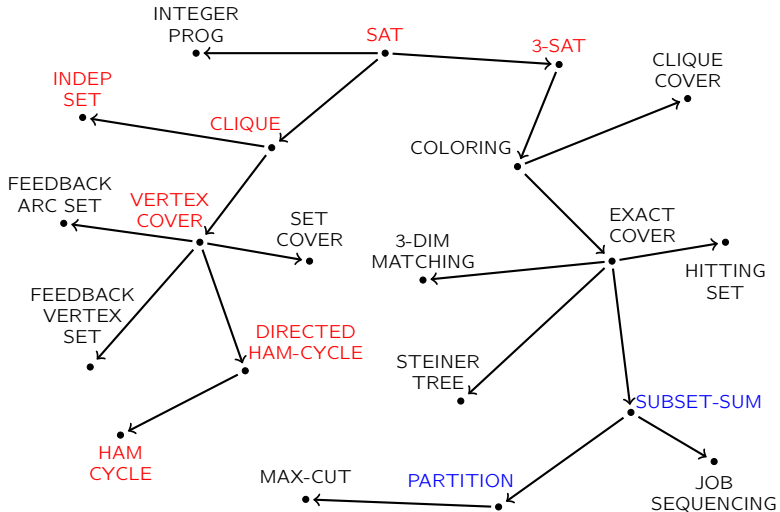
VL-17: Jenseits von P und NP

(Berechenbarkeit und Komplexität, WS 2024)

Peter Rossmanith

WS 2024, RWTH

Wdh.: Landkarte mit Karp's 20 Reduktionen



Wdh.: SUBSET-SUM

Problem: SUBSET-SUM

Eingabe: Positive ganze Zahlen $a_1, \dots, a_n$; eine ganze Zahl b

Frage: Existiert eine Indexmenge $I \subseteq \{1, \dots, n\}$ mit $\sum_{i \in I} a_i = b$?

Satz

SUBSET-SUM ist NP-vollständig.

Satz

PARTITION ist NP-vollständig.

Knapsack ist NP-vollständig.

Bin Packing ist NP-vollständig.

Wdh.: Pseudo-polynomiell versus Stark NP-schwer (1)

- Algorithmus A löst Problem X in **pseudo-polynomieller** Zeit, falls die Laufzeit von A auf Instanzen I von X polynomiell in $|I|$ und $\text{Number}(I)$ beschränkt ist.
- Ein Entscheidungsproblem X ist **stark NP-schwer**, wenn es ein Polynom $q : \mathbb{N} \rightarrow \mathbb{N}$ gibt, sodass die Restriktion von X auf Instanzen I mit $\text{Number}(I) \leq q(|I|)$ NP-schwer ist.

Satz

Es sei X ein stark NP-schweres Entscheidungsproblem.
Falls X pseudo-polynomiell lösbar ist, so gilt $P=NP$.

Also: Pseudo-polynomiell und stark NP-schwer schliessen einander aus
(unter der Annahme $P \neq NP$)

Wdh.: Pseudo-polynomiell versus Stark NP-schwer (2)

Die folgende Tabelle fasst einige unserer Resultate und Beobachtungen unter der Annahme $P \neq NP$ zusammen:

Problem	pseudo-poly		
	NP-schwer	stark NP-schwer	lösbar
SAT	Ja	Ja	Nein
CLIQUE	Ja	Ja	Nein
Ham-Cycle	Ja	Ja	Nein
TSP	Ja	Ja	Nein
SUBSET-SUM	Ja	Nein	Ja
Knapsack	Ja	Nein	Ja
PARTITION	Ja	Nein	Ja
THREE-PARTITION	Ja	Ja	Nein
Bin Packing	Ja	Ja	Nein

Vorlesung VL-17

Jenseits von P und NP

- Die Komplexitätsklasse coNP
- Zwischen P und NPC: NP-intermediate
- Die Komplexitätsklassen EXPTIME und PSPACE
- Und noch einmal: P versus NP

Die Komplexitätsklasse coNP

Definition: Klasse NP (zur Erinnerung)

Ein Entscheidungsproblem $X \subseteq \Sigma^*$ liegt in der Klasse **NP**,
wenn für jede **JA**-Instanz $x \in X$ ein polynomiell langes Zertifikat y
existiert, das in polynomieller Zeit verifiziert werden kann.

Die Klasse coNP

Definition: Klasse NP (zur Erinnerung)

Ein Entscheidungsproblem $X \subseteq \Sigma^*$ liegt in der Klasse **NP**, wenn für jede **JA**-Instanz $x \in X$ ein polynomiell langes Zertifikat y existiert, das in polynomieller Zeit verifiziert werden kann.

Definition: Klasse coNP (neu)

Ein Entscheidungsproblem $X \subseteq \Sigma^*$ liegt in der Klasse **coNP**, wenn für jede **NEIN**-Instanz $x \notin X$ ein polynomiell langes Zertifikat y existiert, das in polynomieller Zeit verifiziert werden kann.

Definition: Klasse NP (zur Erinnerung)

Ein Entscheidungsproblem $X \subseteq \Sigma^*$ liegt in der Klasse **NP**, wenn für jede **JA**-Instanz $x \in X$ ein polynomiell langes Zertifikat y existiert, das in polynomieller Zeit verifiziert werden kann.

Definition: Klasse coNP (neu)

Ein Entscheidungsproblem $X \subseteq \Sigma^*$ liegt in der Klasse **coNP**, wenn für jede **NEIN**-Instanz $x \notin X$ ein polynomiell langes Zertifikat y existiert, das in polynomieller Zeit verifiziert werden kann.

Intuition:

- Wenn X in NP, dann gibt es für **JA**-Instanzen $x \in X$ kurze und einfach zu überprüfende **Beweise**.
- Wenn X in coNP, dann gibt es für **NEIN**-Instanzen $x \notin X$ kurze und einfach zu überprüfende **Widerlegungen**.

Beispiel (1)

Problem: Non-Hamiltonkreis (Non-Ham-Cycle)

Eingabe: Ein ungerichteter Graph $G = (V, E)$

Frage: Besitzt G **keinen** Hamiltonkreis?

Frage: Wie sieht das coNP-Zertifikat für Non-Ham-Cycle aus?

Beispiel (2)

Problem: Unsatisfiability (UNSAT)

Eingabe: Eine Boole'sche Formel φ in CNF über der Boole'schen Variablenmenge $X = \{x_1, \dots, x_n\}$

Frage: Gibt es **keine** Wahrheitsbelegung für X , die φ erfüllt?

Problem: TAUTOLOGY

Eingabe: Eine Boole'sche Formel φ in **DNF** über der Boole'schen Variablenmenge $X = \{x_1, \dots, x_n\}$

Frage: Wird φ von **allen** Wahrheitsbelegungen von X erfüllt?

Frage: Wie sehen coNP-Zertifikate für UNSAT und TAUTOLOGY aus?

Beispiel (3a): Lineare Programmierung

Ein primales Lineares Programm (P):

$$\begin{array}{ll} \max & \sum_{j=1}^n c_j x_j \\ \text{s.t.} & \sum_{j=1}^n a_{ij} x_j \leq b_i \quad \text{für } i = 1, \dots, m \\ & x_j \geq 0 \end{array}$$

Das entsprechende duale Lineare Programm (D):

$$\begin{array}{ll} \min & \sum_{i=1}^m b_i y_i \\ \text{s.t.} & \sum_{i=1}^m a_{ij} y_i \geq c_j \quad \text{für } j = 1, \dots, n \\ & y_i \geq 0 \end{array}$$

Satz (Starker Dualitätssatz)

Wenn beide LPs zulässige Lösungen haben,
so haben beide den selben optimalen Zielfunktionswert.

Beispiel (3b): Lineare Programmierung

Primal= “ $\max cx$ s.t. $Ax \leq b$ ”

Dual= “ $\min by$ s.t. $yA \geq c$ ”

Problem: Lineare Programmierung (LP)

Eingabe: Eine reelle $m \times n$ Matrix A ; Vektoren $b \in \mathbb{R}^m$ und $c \in \mathbb{R}^n$;
eine Schranke $\gamma \in \mathbb{R}$

Frage: Existiert ein Vektor $x \in \mathbb{R}^n$, der $Ax \leq b$ und $x \geq 0$ erfüllt,
und dessen Zielfunktionswert $cx \geq \gamma$ ist?

Beispiel (3b): Lineare Programmierung

Primal= “ $\max cx$ s.t. $Ax \leq b$ ”

Dual= “ $\min by$ s.t. $yA \geq c$ ”

Problem: Lineare Programmierung (LP)

Eingabe: Eine reelle $m \times n$ Matrix A ; Vektoren $b \in \mathbb{R}^m$ und $c \in \mathbb{R}^n$;
eine Schranke $\gamma \in \mathbb{R}$

Frage: Existiert ein Vektor $x \in \mathbb{R}^n$, der $Ax \leq b$ und $x \geq 0$ erfüllt,
und dessen Zielfunktionswert $cx \geq \gamma$ ist?

Beobachtung

LP liegt in NP.

NP-Zertifikat = Vektor x fürs primale LP mit $cx \geq \gamma$

Beispiel (3b): Lineare Programmierung

Primal= “ $\max cx$ s.t. $Ax \leq b$ ”

Dual= “ $\min by$ s.t. $yA \geq c$ ”

Problem: Lineare Programmierung (LP)

Eingabe: Eine reelle $m \times n$ Matrix A ; Vektoren $b \in \mathbb{R}^m$ und $c \in \mathbb{R}^n$;
eine Schranke $\gamma \in \mathbb{R}$

Frage: Existiert ein Vektor $x \in \mathbb{R}^n$, der $Ax \leq b$ und $x \geq 0$ erfüllt,
und dessen Zielfunktionswert $cx \geq \gamma$ ist?

Beobachtung

LP liegt in NP.

NP-Zertifikat = Vektor x fürs primale LP mit $cx \geq \gamma$

Beobachtung

LP liegt in coNP.

coNP-Zertifikat = Vektor y fürs duale LP mit $by < \gamma$

Beispiel (3c): Lineare Programmierung

Zusammenfassung

LP liegt in $NP \cap coNP$.

Anmerkung: Das war bereits in den 1950er Jahren bekannt.

Satz (Leonid Genrikhovich Khachiyan, 1979)

LP liegt in P .

Anmerkung: Khachiyan entwickelte die **Ellipsoid-Methode** für LPs.

coNP-Vollständigkeit

coNP-Vollständigkeit (1)

Definition: NP-vollständig (zur Erinnerung)

Ein Entscheidungsproblem X ist **NP**-vollständig, wenn $X \in \mathbf{NP}$ und alle $Y \in \mathbf{NP}$ polynomiell auf X reduzierbar sind.

Definition: coNP-vollständig (neu)

Ein Entscheidungsproblem X ist **coNP**-vollständig, wenn $X \in \mathbf{coNP}$ und alle $Y \in \mathbf{coNP}$ polynomiell auf X reduzierbar sind.

Intuition:

- X ist NP-vollständig, wenn es zu den schwierigsten Problemen in NP gehört.
- X ist coNP-vollständig, wenn es zu den schwierigsten Problemen in coNP gehört.

Satz

Non-Ham-Cycle, UNSAT und TAUTOLOGY sind coNP-vollständig.

Beweis: Übung.

Satz

Non-Ham-Cycle, UNSAT und TAUTOLOGY sind coNP-vollständig.

Beweis: Übung.

Satz

Wenn das Entscheidungsproblem X NP-vollständig ist,
so ist das komplementäre Problem $\overline{X}$ coNP-vollständig.

Komplementäres Problem:

Ja-Instanzen von X werden zu Nein-Instanzen von $\overline{X}$, und
Nein-Instanzen von X werden zu Ja-Instanzen von $\overline{X}$

Satz

$$P \subseteq NP \cap \text{coNP}$$

Beweis: $P = \text{coP}$

coNP versus NP (1)

Viele Informatiker denken, dass $NP \neq coNP$ gilt.

Satz

Wenn $coNP$ ein NP-vollständiges Problem X enthält, dann $NP = coNP$.

coNP versus NP (1)

Viele Informatiker denken, dass $NP \neq coNP$ gilt.

Satz

Wenn $coNP$ ein NP-vollständiges Problem X enthält, dann $NP = coNP$.

Beweis:

- $X \in NPC$ impliziert: $L \leq_p X$ für alle $L \in NP$

coNP versus NP (1)

Viele Informatiker denken, dass $NP \neq coNP$ gilt.

Satz

Wenn $coNP$ ein NP-vollständiges Problem X enthält, dann $NP = coNP$.

Beweis:

- $X \in NPC$ impliziert: $L \leq_p X$ für alle $L \in NP$
- $X \in NPC$ impliziert: $\bar{L} \leq_p \bar{X}$ für alle $L \in NP$

coNP versus NP (1)

Viele Informatiker denken, dass $NP \neq coNP$ gilt.

Satz

Wenn $coNP$ ein NP-vollständiges Problem X enthält, dann $NP = coNP$.

Beweis:

- $X \in NPC$ impliziert: $L \leq_p X$ für alle $L \in NP$
- $X \in NPC$ impliziert: $\bar{L} \leq_p \bar{X}$ für alle $L \in NP$
- $X \in NPC$ impliziert: $K \leq_p \bar{X}$ für alle $K \in coNP$

coNP versus NP (1)

Viele Informatiker denken, dass $NP \neq coNP$ gilt.

Satz

Wenn $coNP$ ein NP-vollständiges Problem X enthält, dann $NP = coNP$.

Beweis:

- $X \in NPC$ impliziert: $L \leq_p X$ für alle $L \in NP$
- $X \in NPC$ impliziert: $\bar{L} \leq_p \bar{X}$ für alle $L \in NP$
- $X \in NPC$ impliziert: $K \leq_p \bar{X}$ für alle $K \in coNP$
- Ergo: Alle $K \in coNP$ sind auf $\bar{X} \in NP$ reduzierbar.

coNP versus NP (1)

Viele Informatiker denken, dass $NP \neq coNP$ gilt.

Satz

Wenn $coNP$ ein NP-vollständiges Problem X enthält, dann $NP = coNP$.

Beweis:

- $X \in NPC$ impliziert: $L \leq_p X$ für alle $L \in NP$
- $X \in NPC$ impliziert: $\bar{L} \leq_p \bar{X}$ für alle $L \in NP$
- $X \in NPC$ impliziert: $K \leq_p \bar{X}$ für alle $K \in coNP$
- Ergo: Alle $K \in coNP$ sind auf $\bar{X} \in NP$ reduzierbar.
- Ergo: Alle $K \in coNP$ liegen in NP . Q.E.D.

coNP versus NP (1)

Viele Informatiker denken, dass $NP \neq coNP$ gilt.

Satz

Wenn $coNP$ ein NP-vollständiges Problem X enthält, dann $NP = coNP$.

Beweis:

- $X \in NPC$ impliziert: $L \leq_p X$ für alle $L \in NP$
- $X \in NPC$ impliziert: $\bar{L} \leq_p \bar{X}$ für alle $L \in NP$
- $X \in NPC$ impliziert: $K \leq_p \bar{X}$ für alle $K \in coNP$
- Ergo: Alle $K \in coNP$ sind auf $\bar{X} \in NP$ reduzierbar.
- Ergo: Alle $K \in coNP$ liegen in NP . Q.E.D.

Daraus erhalten wir das folgende Werkzeug:

“ X NP-vollständig”	ist Evidenz für	“ $X \notin coNP$ ”
“ X coNP-vollständig”	ist Evidenz für	“ $X \notin NP$ ”

coNP versus NP (2)

Wir erhalten das folgende Werkzeug:

“ X NP-vollständig”	ist Evidenz für	“ $X \notin \text{coNP}$ ”
“ X coNP-vollständig”	ist Evidenz für	“ $X \notin \text{NP}$ ”

Ham-Cycle ist NP-vollständig.

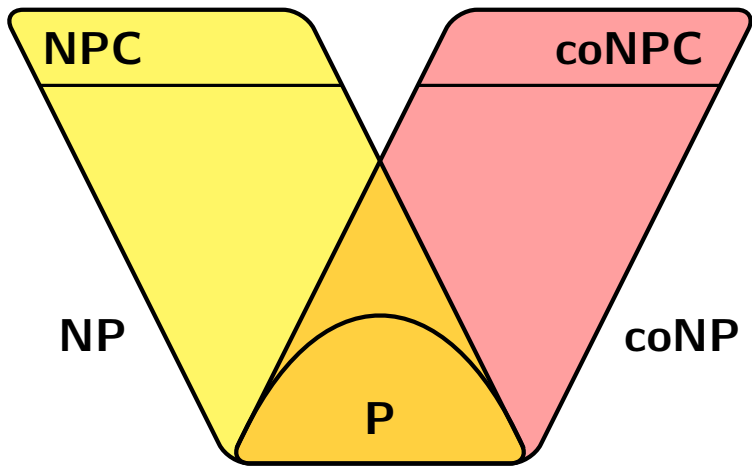
Ham-Cycle hat gute Zertifikate für Ja-Instanzen.

Ergo: Ham-Cycle hat (höchstwahrscheinlich) keine guten Zertifikate für Nein-Instanzen.

SAT ist NP-vollständig.

SAT hat gute Zertifikate für Ja-Instanzen.

Ergo: SAT hat (höchstwahrscheinlich) keine guten Zertifikate für Nein-Instanzen.



Der Durchschnitt von coNP und NP

Viele **Mathematiker** denken, dass $NP \cap coNP = P$ gilt.

Beispiel

LP in $NP \cap coNP$ war seit den 1950er Jahren bekannt.
Erst 1979 wurde ein polynomieller Algorithmus für LP gefunden.

Beispiel

PRIMZAHL in $NP \cap coNP$ war seit den 1970er Jahren bekannt.
Erst 2002 wurde ein polynomieller Algorithmus für Primzahl-Test gefunden.

Beispiel

PARITY-GAME in $NP \cap coNP$ ist seit den 1990er Jahren bekannt.
Ob allerdings PARITY-GAME in P liegt, ist ein offenes Problem.

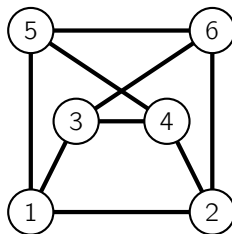
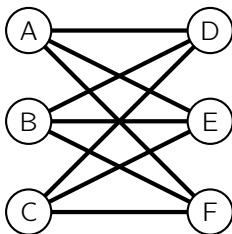
Zwischen P und NPC: NP-intermediate

Das Graphisomorphieproblem (1)

Definition

Zwei Graphen $G_1 = (V_1, E_1)$ und $G_2 = (V_2, E_2)$ sind **isomorph**, wenn es eine Bijektion $f : V_1 \rightarrow V_2$ gibt, die Adjazenzen und Nicht-Adjazenzen erhält.

Eine solche Bijektion heisst **Isomorphismus**.



Das Graphisomorphieproblem (2)

Problem: GRAPH-ISOMORPHUS

Eingabe: Zwei ungerichtete Graphen G_1 und G_2

Frage: Gibt es einen Isomorphismus von G_1 nach G_2 ?

Satz

GRAPH-ISOMORPHUS liegt in NP.

Beweis: Verwende Isomorphismus als Zertifikat.

Das Graphisomorphieproblem (3)

Folgende Fragen sind zur Zeit noch ungelöst:

- Liegt GRAPH-ISOMORPHUS in P?
- Ist GRAPH-ISOMORPHUS NP-vollständig?
- Liegt GRAPH-ISOMORPHUS in coNP?

Das Graphisomorphieproblem (4)

Satz (László Babai, 2016)

GRAPH-ISOMORPHUS auf Graphen mit n Knoten
kann in $2^{p(\log n)}$ Zeit gelöst werden (wobei p ein Polynom ist).

Babai's Algorithmus verwendet schwere algebraische Werkzeuge
(algorithmische und strukturelle Theorie der Permutationsgruppen).

NP-intermediate (1)

Definition

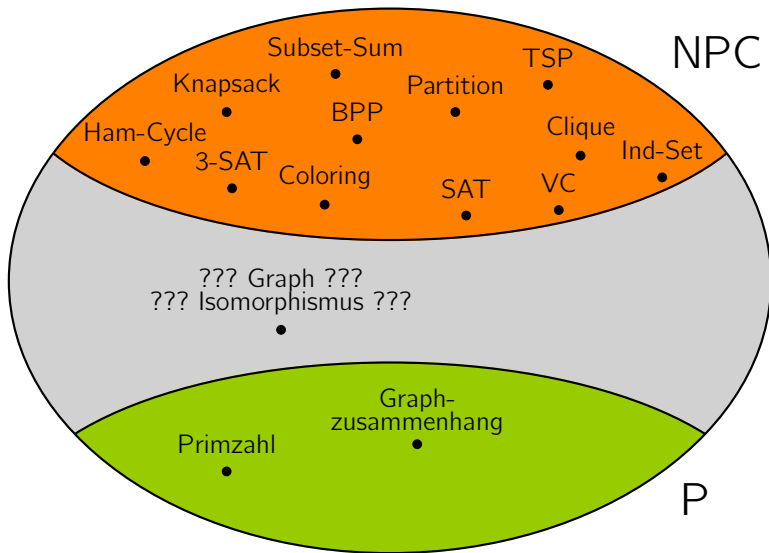
Ein Entscheidungsproblem $L \subseteq \Sigma^*$ heisst **NP-intermediate**, wenn $L \in NP$ und wenn sowohl $L \notin P$ als auch $L \notin NPC$ gilt.

Satz von Ladner (ohne Beweis)

Wenn $P \neq NP$ gilt,
dann existieren Probleme, die NP-intermediate sind.

Viele Informatiker denken,
dass das GRAPH-ISOMORPHUS Problem NP-intermediate ist.

NP-intermediate (2)



Warnung: Dieser Abbildung liegt die Annahme $P \neq NP$ zu Grunde.

Die Komplexitätsklassen PSPACE und EXPTIME

Die Klasse PSPACE (1)

Definition: Komplexitätsklasse PSPACE

PSPACE ist die Klasse aller Entscheidungsprobleme, die durch eine **DTM** M entschieden werden, deren Worst Case Speicherplatzbedarf durch $q(n)$ mit einem Polynom q beschränkt ist.

Definition: Komplexitätsklasse NPSPACE

NPSPACE ist die Klasse aller Entscheidungsprobleme, die durch eine **NTM** M entschieden werden, deren Worst Case Speicherplatzbedarf durch $q(n)$ mit einem Polynom q beschränkt ist.

Die Klasse PSPACE (2)

Satz von Savitch (ohne Beweis)

$\text{PSPACE} = \text{NPSPACE}$

Satz von Savitch (ohne Beweis)

$PSPACE = NPSPACE$

Wie verhält sich PSPACE zu NP?

Da sich der Kopf einer Turingmaschine in einem Schritt nur um eine Position bewegen kann gilt: $NP \subseteq NPSPACE = PSPACE$

Die Klasse PSPACE (3)

Problem: QUANTIFIED-SAT (Q-SAT)

Eingabe: Eine Boole'sche Formel φ in CNF über der Boole'schen Variablenmenge $\{x_1, \dots, x_n, y_1, \dots, y_n\}$

Frage: $\exists x_1 \forall y_1 \exists x_2 \forall y_2 \exists x_3 \forall y_3 \dots \exists x_n \forall y_n \varphi$

Satz

Q-SAT liegt in PSPACE.

Anmerkung: Q-SAT ist PSPACE-vollständig.

Die Klasse EXPTIME (1)

Definition: Komplexitätsklasse EXPTIME

EXPTIME ist die Klasse aller Entscheidungsprobleme, die durch eine DTM M entschieden werden, deren Worst Case Laufzeit durch $2^{q(n)}$ mit einem Polynom q beschränkt ist.

- Laufzeit-Beispiele: $2^{\sqrt{n}}$, 2^n , 3^n , $n \cdot 2^n$, $n!$, n^n
- Aber nicht: 2^{2^n} und n^{n^n}

Wie verhält sich EXPTIME zu PSPACE?

- Bei einer Speicherplatzbeschränkung $s(n)$ gibt es nur $2^{O(s(n))}$ viele verschiedenen Konfigurationen für eine Turingmaschine. Daher ist die Rechenzeit durch $2^{O(s(n))}$ beschränkt.
- Die Probleme in PSPACE können deshalb in Zeit $2^{p(n)}$ gelöst werden: $\text{PSPACE} \subseteq \text{EXPTIME}$

Die Klasse EXPTIME (3)

Problem: k -Schritt-HALTEPROBLEM

Eingabe: Eine deterministische Turingmaschine M ; eine ganze Zahl k

Frage: Wenn M mit leerem Band gestartet wird, hält M dann nach höchstens k Schritten an?

Die Zahl k ist binär (oder dezimal) kodiert.

Satz

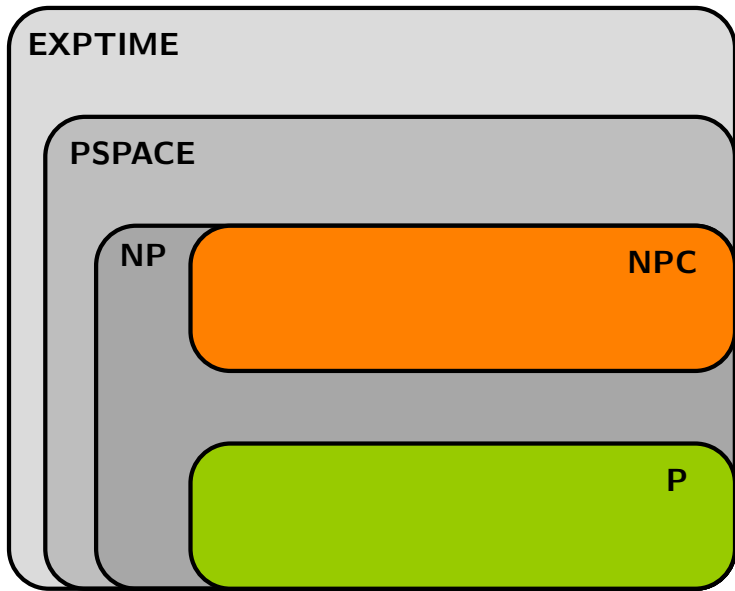
Das k -Schritt-HALTEPROBLEM liegt in EXPTIME.

Anmerkung: Das k -Schritt-HALTEPROBLEM ist EXPTIME-vollständig.

Wir haben gezeigt:

$$P \subseteq NP \subseteq PSPACE \subseteq EXPTIME$$

- Es ist nicht bekannt, welche dieser Inklusionen strikt sind. Möglicherweise gilt $P = PSPACE$ oder $NP = EXPTIME$.
- Wir wissen allerdings, dass $P \neq EXPTIME$ gilt.
(Das folgt aus dem so-geannten **Zeithierarchiesatz**.)
- Wir vermuten, dass alle drei Inklusionen strikt sind.



Noch einmal: P versus NP

Zusammenfassung der Vorlesungen über P und NP:

Frage: Wie löse ich eine SAT Instanz mit n Variablen?

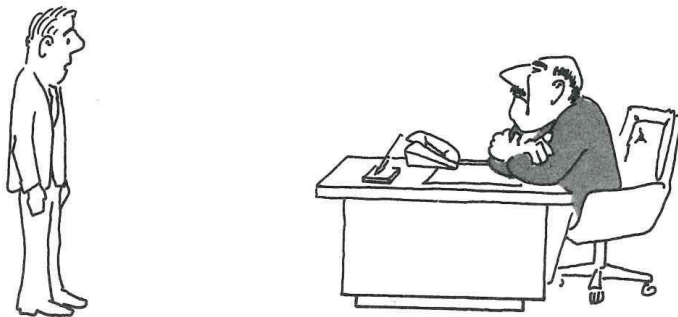
Antwort: Probiere alle 2^n Wahrheitsbelegungen durch.

Frage: Geht das auch schneller?

Antwort: Das wissen wir nicht.

P versus NP im echten Leben (1)

(Aus dem Buch "Computers and Intractability von M.R.Garey und D.S.Johnson)



"I can't find an efficient algorithm, I guess I'm just too dumb."

P versus NP im echten Leben (2)

(Aus dem Buch "Computers and Intractability von M.R.Garey und D.S.Johnson)



"I can't find an efficient algorithm, because no such algorithm is possible!"

P versus NP im echten Leben (3)

(Aus dem Buch "Computers and Intractability von M.R.Garey und D.S.Johnson)

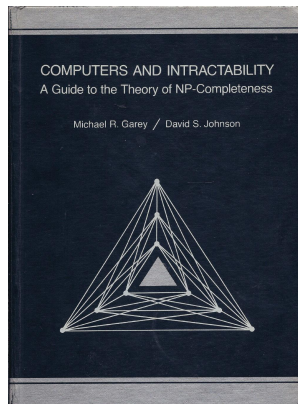


"I can't find an efficient algorithm, but neither can all these famous people."

Das Buch von Garey and Johnson

Michael R. Garey and David S. Johnson:
“Computers and Intractability: A Guide to
the Theory of NP-Completeness”, 1979

- The first book “exclusively on the theory of NP-completeness and computational intractability”
- Appendix enthält Liste mit 300 NP-vollständigen Problems
- Meist zitiertes Buch der Informatik



P versus NP: Die Anfänge

*"The classes of problems which are respectively known and not known to have good algorithms are of great theoretical interest.
I conjecture that there is no good algorithm for the Travelling Salesman Problem. My reasons are the same as for any mathematical conjecture:
(1) It is a legitimate mathematical possibility, and
(2) I do not know."*

– Jack Edmonds, 1966

"We seem to be missing even the most basic understanding of the nature of its difficulty. All approaches tried so far probably (in some cases, provably) have failed. In this sense $P \neq NP$ is different from many other major mathematical problems on which a gradual progress was being constantly done (sometimes for centuries) whereupon they yielded, either completely or partially."

– Alexander Razborov, 2002

“In my opinion this shouldn't really be a hard problem; it's just that we came late to this theory, and haven't yet developed any techniques for proving computations to be hard. Eventually, it will just be a footnote in the books.”

– John Horton Conway, 2002

"I lean towards $P \neq NP$, but I would not bet anything significant on it. I think it is premature to conjecture which way the question will be resolved. We cannot even rule out linear time SAT algorithms. Common sense says that the universe is simply not nice enough that P should equal NP . But I don't know how to justify that formally."

– Ryan Williams, 2012